

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет безопасности информационных технологий

Образовательная программа: 10.03.01 Технологии защиты информации

Направление (специальность): Комплексная защита объектов информатизации

ОТЧЕТ

на производственную, проектно-технологическую практику

Тема задания:

Тестирование разработанного механизма защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров

Обучающийся:

студент группы N3441 Александров Юрий Борисович

Согласовано:

Руководитель практики от профильной организации Тарасов Павел Андреевич, ООО «Мультиагентные системы», инженер-программист

(подпись)

Руководитель практики от Университета ИТМО Руководитель ВКР, должность

(подпись)

Практика пройдена с оценкой _____

Дата 29.04.2026

Санкт-Петербург

2026

Содержание

Введение.....	3
1. Угрозы, связанные с весами моделей нейронных сетей.....	4
2. Существующие решения для сохранения конфиденциальности весов моделей нейронных сетей.....	5
3. Разработка механизма защиты весов нейронных сетей от несанкционированного доступа.....	11
Заключение.....	14
Список литературы.....	15

Введение

Цель работы – протестировать разработанный механизм защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров.

Для достижения цели необходимо выполнить следующие **задачи**:

- описать разработанный метод защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров;
- провести тестирование эффективности разработанного метода на обеспечение устойчивости к краже модели нейронной сети;
- провести сравнительный анализ модели нейронной сети с классической архитектурой и модели с предложенной архитектурой, реализующей механизм защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров;
- рассмотреть возможности применения предложенного метода защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров в промышленных решениях;
- оформить отчет по проделанной работе.

Данная работа является частью производственной, проектно-технологической практики, проходившей в Университете ИТМО в статусе практиканта с «03» марта 2025 г. по «25» апреля 2025 г.

1 МЕХАНИЗМ ЗАЩИТЫ ВЕСОВ МОДЕЛЕЙ НЕЙРОННЫХ СЕТЕЙ

Искусственный интеллект стал неотъемлемой частью жизни каждого современного человека. Невозможно не замечать ежегодный рост и расширение областей применения систем, реализующих методы машинного обучения, для решения широкого спектра прикладных задач, куда входят:

- компьютерное зрение[1];
- обработка текста[2];
- управление беспилотными автомобилями[3].

Стоит отметить, что нейронные сети все чаще применяются в областях, требующих соблюдения особой конфиденциальности, таких как:

- медицинская[4];
- правоохранительная[5].

В связи с заинтересованностью в обеспечении безопасности моделей машинного обучения мной был разработан метод защиты весов моделей нейронных сетей.

Мной был разработан метод защиты весов моделей нейронных сетей, который основан на внедрении в архитектуру нейронной сети скрытого входного параметра – ключа.

Ключ – специальный параметр, который подается на вход модели с основными данными и без которого невозможна корректная работа модели нейронной сети.

Ключ выступает в роли внутреннего механизма валидации доступа к модели нейронной сети.

При использовании ключевого метода защиты, работоспособность модели напрямую зависит от знания пользователем ключа:

- при использовании верного ключа модель способна выдавать достоверные результаты;
- при использовании неверного ключа модель выдает непредсказуемые результаты;
- без подачи какого-либо ключа модель неработоспособна.

Таким образом, даже если злоумышленник получит прямой доступ к модели, он не сможет воспользоваться ей или скопировать ее.

Процесс внедрения ключевого входа требует внесения изменений в архитектуру модели нейронной сети. Это достигается путем добавления дополнительного входа в

модель и дальнейшего объединения основного входа и ключа.

Основные характеристики ключевого входа:

- внедряется непосредственно в архитектуру модели;
- позволяет создать скрытую зависимость между параметрами ключа и активациями внутренних слоёв;
- шифрует структуру модели и ее веса;
- создает зависимость между правильным значением ключа и рабочим состоянием модели уже на этапе обучения.

Ключ предлагается реализовать в виде тензора.

Его генерация может осуществляться как случайно, так и на основе следующих индивидуальных признаков (Таблица 3):

- биометрические данные пользователя;
- уникальные параметры устройства.

Таблица 3 - Типы ключей по способу генерации

Тип ключа	Данные для генерации
Случайный тензор	Случайная генерация при и инициализации модели
Биометрический	Отпечаток пальца, голос
Аппаратный	MAC-адрес, серийный номер процессора

Ключевой вход может применяться ко многим видам нейронных сетей, таким как:

- полносвязные сети;
- сверточные нейронные сети[33];
- рекуррентные нейронные сети[34];
- генеративные состязательные сети[35].

Таким образом, ключевой вход представляет собой универсальный механизм, который может быть внедрен в любую современную нейронной сети без существенного изменения ее архитектуры и логики.

Отдельно стоит отметить влияние ключа на веса модели нейронной сети, так как это является отличительной особенностью данного метода.

На этапе обучения ключ подается на вход вместе с основными данными и участвует в формировании градиентов, а это уже напрямую влияет на значения весов нейронной сети

Следовательно, при обучении модели с ключевым входом, веса автоматически шифруются.

Архитектура нейронной сети[36] отображается следующей формулой (1): $f: R^n \rightarrow R^m$, $f(x; \theta)$, (1)

где $x \in R^n$ – входной вектор признаков; θ – параметры модели (веса и смещения); f – отображение, реализуемое нейросетью.

Далее мы можем наблюдать схематическое изображение классической модели нейронной сети (Рисунок 2).

В предлагаемом методе защиты весов моделей нейронных сетей от несанкционированного доступа входной вектор модифицируется до(2):

$$x' = [x, K], K \in R^k, (2)$$

где K – тензор ключа, объединяемый с основными данными (через конкатенацию, суммирование или более сложную трансформацию).

Модель принимает вид(3):

$$f': R^{n+k} \rightarrow R^m, f'([x, K]; \theta'). (3)$$

Также мы можем наблюдать схематическое изображение модели нейронной сети с ключевым входом (Рисунок 3).



- реализация модели нейронной сети;
- модификация архитектуры нейронной сети с внедрением ключевого входа в архитектуру.

Реализованная нейронная сеть решает задачу классификации изображений кошек и собак.

Для разработки и обучения модели нейронной сети были выбраны:

- фреймворк PyTorch: удобен для реализации сверточной нейронной сети;
- облачная среда разработки Colab: предоставляет доступ к аппаратным ускорителям(GPU/TPU), что значительно ускоряет процесс обучения, что особенно актуально при обучении модели на большом датасете;
- открытые датасеты с платформы Kaggle, содержащие изображения кошек и собак, классифицированные по соответствующим папкам, которые необходимы для обучения модели.

Для обучения обеих моделей использовались одинаковые датасеты, что позволит далее провести объективное сравнение двух реализованных архитектур.

2 ТЕСТИРОВАНИЕ

Для оценки эффективности предложенного метода защиты весов моделей нейронных сетей от несанкционированного доступа на основе скрытых входных параметров была проведена серия экспериментов, направленных на проверку эффективности в противостоянии кражам моделей.

В том числе была проверена работоспособность и точность результатов при:

- использовании корректного ключа;
- использовании неверного ключа;
- удалении ключа.

Далее рассмотрим результаты проведенных экспериментов (Таблица 4).

Таблица 4 - Результаты экспериментов

Режим проверки	Точность(%)
С корректным ключом	96,7
С неверным ключом	34,2

Более того, тестирование подтвердило, что без ключа данная модель нейронной сети не работоспособна.

Таким образом, мы можем наблюдать, что использование ключевой архитектуры демонстрирует ожидаемые результаты и показывает высокую эффективность.

При использовании неверного ключа наблюдается абсолютно случайных характер ответов, что подтверждает устойчивость ключевой архитектуры к краже.

Как было сказано ранее, предложенный метод должен обеспечивать не только защиту модели, но и конкретно защиту весов, то есть их шифрование.

Для проверки данного тезиса было взято два файла с весами:

- файл с весами модели с классической архитектурой;
- файл с весами модели с модифицированной архитектурой.

Таблица 5 - Сравнение классической модели и модели с ключевым входом

Параметр	Классическая архитектура модели	Модель с ключевой архитектурой
Точность на тестовой выборке	96,7 %	96,7 %
Устойчивость к краже модели	Не обеспечивается	Высокая

Таблица 5 - Сравнение классической модели и модели с ключевым входом

Параметр	Классическая архитектура модели	Модель с ключевой архитектурой
Устойчивость к краже весов	Не обеспечивается	Высокая
Устойчивость к краже обучающего датасета	Не обеспечивается	Высокая
Усложнение архитектуры	-	Незначительное

По результатам сравнения двух файлов, было выявлено, что сходство между ними составляет 6%, что говорит о том, что файл с весами значительно изменен. Далее проведем сравнение модели с классической архитектурой и модели с реализованным ключевым входом (Таблица 5).

Таким образом, модель с ключевым входом отличается от классической нейронной сети устойчивостью к:

- модели;
- весов;
- обучающего датасета.

Также стоит отметить, что добавление ключевого входа незначительно влияет на скорость обучения (Таблица 6), что делает данный метод удобным для разработчиков.

Таблица 4 - Сравнение моделей по времени обучения

Архитектура модели	Среднее время обучения (мин)
Классическая	8
С ключевым входом	10

Таким образом, предложенный и реализованный метод главным образом:

- обеспечивает высокую устойчивость к краже модели;
- не снижает точности вычислений;
- незначительно усложняет архитектуру.

Это делает подход перспективным для защиты нейронных сетей.

3 ПРИМЕНЕНИЕ В ИНДУСТРИАЛЬНЫХ РЕШЕНИЯХ

Предложенный метод защиты весов моделей нейронных сетей от несанкционированного доступа на основе скрытых входных параметров является практически значимым и может быть внедрен в различные промышленные решения.

Далее подробнее рассмотрим некоторые из них.

Во многих устройствах интернета вещей[37], которые могут быть физически украдены, модели нейронных сетей размещаются локально, например, на:

- микроконтроллерах;
- камерах с локальной обработкой изображений и видео;
- медицинских портативных устройствах, таких как мониторы сердечного ритма или глюкометры с функциями предсказания;
- голосовых ассистентах и умные колонки, постоянно обрабатывающие аудиосигналы.

Это делает их особенно уязвимыми к извлечению и несанкционированному копированию.

Встроенный механизм ключевого входа позволяет сделать модель нефункциональной без знания ключа.

В данном случае ключ может быть привязан к уникальным характеристикам устройства, таким как:

- серийный номер;
- MAC-адрес;
- уникальный идентификатор микроконтроллер (UID);
- идентификатор процессора (CPU ID).

Современные компании все чаще внедряют системы, которые используют модели нейронных сетей, в свои программные продукты.

В таком случае метод ключевого входа может использоваться, чтобы защитить свои модели от пиратства и нелегального внедрения.

Ключ может быть:

- уникальным для каждого пользователя;
- привязан к конкретному экземпляру или версии программного обеспечения.

Примеры продуктов, в которые может быть внедрена ключевая защита:

- системы анализа документов[38];

- системы поддержки клиентов;
- системы распознавания речи[39].

Интеллектуальные системы являются техническими комплексами, в которых ключевую роль играют модели нейронных сетей, выполняющие следующие функции:

- анализа;
- управление;
- принятие решений.

Безопасность интеллектуальных систем особенно критична, так как они:

- работают в автономном или полуавтономном режиме;
- взаимодействуют с окружающим миром и людьми.

. Внедрение модели с ключевым входом делает невозможным её несанкционированное использование при кражах.

Также, стоит отметить, что применение защищённой модели с ключевым входом повышает доверие пользователей к интеллектуальным системам.

Модель с ключом может внедряться в следующие интеллектуальные системы:

- автопилоты[40];
- промышленные роботы[41];
- беспилотники[42, 43].

В современном мире все чаще встречаются случаи коммерциализации разработок в сфере нейронных сетей и, в том числе, передаче моделей:

- организациям;
- интеграторам;
- разработчикам.

Все сказанное выше сопряжено с рисками несанкционированного и нелегального использования модели, в том числе:

- копирования;
- распространения.

Для защиты от перечисленных угроз можно передавать версию, активируемую уникальным ключом, с возможностью управления доступом по:

- лицензии;
- времени действия.

Данный метод может применяться в:

- В2В-продуктах: заказчик покупает права на использование модели нейронной сети для встраивания в собственный продукт;
- OEM-партнерстве: производитель аппаратного обеспечения получает модель для интеграцию в собственный продукт.

Таким образом, метод ключевой защиты может применяться для построения защищенной бизнес-модели в сфере продажи готовых нейросетевых решений.

Предложенный метод защиты весов моделей нейронных сетей на основе скрытых входных, показал высокую эффективность в противодействии несанкционированному доступу и устойчивость к попыткам использования модели без знания ключа.

Однако, данный метод имеет ряд потенциальных угроз и может стать объектом атаки со стороны злоумышленника.

Таким образом, предложенный метод защиты весов моделей нейронных сетей на основе скрытых входных параметров (ключевого входа) обладает большим потенциалом для дальнейшего развития.

Предложенный метод уже обеспечивает защиту от несанкционированного доступа и кражи модели, однако с ростом сложности атак он может показать недостаточную эффективность.

В целях повышения устойчивости моделей к взлому, метод ключевого входа в дальнейшем может быть доработан путями рассмотренными далее.

Одним из наиболее перспективных направлений развития является интеграция ключа с криптографическими методами защиты, такими как:

- гомоморфное шифрование;
- Secure Multiparty Computation.

Таким образом, будет реализован двойной уровень защиты:

- на уровне архитектуры;
- на уровне обработки данных.

Другим направлением реализации могут быть динамически изменяющихся ключей, которые смогут автоматически обновляться:

- через определенные промежутки времени;
- в случае подозрительной активности.

Отдельного внимания заслуживает перспектива разработки инструментов для встраивания ключевого механизма в готовые модели.

Основная идея состоит в том, чтобы создать программные решения, которые позволят автоматически модифицировать нейронные сети, добавляя в них ключ.

Это позволит:

- внедрять ключевой вход в уже существующие модели нейронных сетей;
- упростить работу разработчикам, которым необходим реализовать архитектуру с ключевым входом.

Заключение

В ходе проделанной работы был протестирован программный код, реализующий механизм защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров.

Были выполнены следующие **задачи**:

- описан разработанный метод защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров;
- проведено тестирование эффективности разработанного метода на обеспечение устойчивости к краже модели нейронной сети;
- проведено сравнительный анализ модели нейронной сети с классической архитектурой и модели с предложенной архитектурой, реализующей механизм защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров;
- рассмотрены возможности применения предложенного метода защиты от несанкционированного доступа к весам моделей нейронных сетей на основе скрытых входных параметров в индустриальных решениях;
- оформлен отчет по проделанной работе.

Таким образом, задачи выполнены, цель работы достигнута.

Список литературы

1. Madhavaram C. R. et al. The future of automotive manufacturing: Integrating AI, ML, and Generative AI for next-Gen Automatic Cars //International Multidisciplinary Research Journal Reviews-IMRJR. – 2024. – Т. 1. – С. 010103 (дата обращения: 03.05.2025);
2. Papik K. et al. Application of neural networks in medicine-a review //Med Sci Monit. – 1998. – Т. 4. – №. 3. – С. 538-546 (дата обращения: 04.05.2025);
3. Zhao W. et al. Face recognition: A literature survey //ACM computing surveys (CSUR). – 2003. – Т. 35. – №. 4. – С. 399-458 (дата обращения: 05.05.2025);
4. Voulodimos A. et al. Deep learning for computer vision: A brief review //Computational intelligence and neuroscience. – 2018. – Т. 2018. – №. 1. – С. 7068349. (дата обращения: 06.05.2025)
5. McCrudden M. T., Schraw G. Relevance and goal-focusing in text processing //Educational psychology review. – 2007. – Т. 19. – С. 113-139. (дата обращения: 08.05.2025)
6. Madhavaram C. R. et al. The future of automotive manufacturing: Integrating AI, ML, and Generative AI for next-Gen Automatic Cars //International Multidisciplinary Research Journal Reviews-IMRJR. – 2024. – Т. 1. – С. 010103. (дата обращения: 10.05.2025)
7. Papik K. et al. Application of neural networks in medicine-a review //Med Sci Monit. – 1998. – Т. 4. – №. 3. – С. 538-546. (дата обращения: 11.05.2025)
8. Zhao W. et al. Face recognition: A literature survey //ACM computing surveys (CSUR). – 2003. – Т. 35. – №. 4. – С. 399-458. (дата обращения: 14.05.2025)
9. Slattery P. et al. The ai risk repository: A comprehensive meta-review, database, and taxonomy of risks from artificial intelligence //arXiv preprint arXiv:2408.12622. – 2024. (дата обращения: 15.05.2025)
10. Liang J. et al. Model extraction attacks revisited //Proceedings of the 19th ACM Asia Conference on Computer and Communications Security. – 2024. – С. 1231-1245. (дата обращения: 18.05.2025)
11. Wilamowski B. M. Neural network architectures and learning algorithms //IEEE Industrial Electronics Magazine. – 2009. – Т. 3. – №. 4. – С. 56-63. (дата обращения: 22.05.2025)
12. Cao W. et al. A review on neural networks with random weights //Neurocomputing. – 2018. – Т. 275. – С. 278-287. (дата обращения: 24.05.2025)

ПРИЛОЖЕНИЕ

Программная реализация сверточной нейронной сети для классификации изображений кошек и собак.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader
from tqdm import tqdm

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
])

train_dogs_path = '/content/drive/My Drive/data/Dog/'
train_cats_path = '/content/drive/My Drive/data/Cat/'

train_dataset = datasets.ImageFolder('/content/drive/My Drive/data/', transform=transform)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3),
            nn.LeakyReLU(0.2),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3),
            nn.LeakyReLU(0.2),
            nn.MaxPool2d(2),
        )
```



```

self.fc_layers = nn.Sequential(
    nn.Linear(64 * 14 * 14, 128),
    nn.LeakyReLU(0.2),
    nn.Linear(128, 2)
)

def forward(self, x):
    x = self.conv_layers(x)
    x = x.view(x.size(0), -1)
    x = self.fc_layers(x)
    return x

def accuracy(preds, labels):
    _, predicted = torch.max(preds.data, 1)
    return (predicted == labels).float().mean()

model = CNN()

optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
loss_fn = nn.CrossEntropyLoss()

epochs = 8
for epoch in range(epochs):
    total_loss = 0
    total_accuracy = 0

    for sample in tqdm(train_loader):
        imgs, labels = sample[0], sample[1]
        optimizer.zero_grad()

        preds = model(imgs)
        loss = loss_fn(preds, labels)

        loss.backward()

```

```
optimizer.step()

total_loss += loss.item()
total_accuracy += accuracy(preds, labels)

avg_loss = total_loss / len(train_loader)
avg_accuracy = total_accuracy / len(train_loader)

print(f'Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, Accuracy: {avg_accuracy:.4f}')
```

ПРИЛОЖЕНИЕ Б

Программная реализация сверточной нейронной сети для классификации изображений кошек и собак с реализованным ключевым доступом.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader
from tqdm import tqdm

# Трансформации
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
])

# Датасеты
train_dataset = datasets.ImageFolder('/content/drive/My Drive/data/', transform=transform)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)

# Реализация архитектуры модели нейронной сети с ключом
class CNNWithKey(nn.Module):
    def __init__(self, key_size=16):
        super(CNNWithKey, self).__init__()
        self.key_size = key_size

    self.conv_layers = nn.Sequential(
        nn.Conv2d(3, 32, kernel_size=3),
        nn.LeakyReLU(0.2),
        nn.MaxPool2d(2),
        nn.Conv2d(32, 64, kernel_size=3),
        nn.LeakyReLU(0.2),
        nn.MaxPool2d(2),
```

```

)

self.fc1 = nn.Linear(64 * 14 * 14 + key_size, 128)
self.relu = nn.LeakyReLU(0.2)
self.fc2 = nn.Linear(128, 2)

def forward(self, x, key):
    x = self.conv_layers(x)
    x = x.view(x.size(0), -1)
    x = torch.cat((x, key), dim=1) # Добавляем ключ
    x = self.relu(self.fc1(x))
    x = self.fc2(x)
    return x

# Точность
def accuracy(preds, labels):
    _, predicted = torch.max(preds.data, 1)
    return (predicted == labels).float().mean()

# Инициализация модели
key_size = 16
model = CNNWithKey(key_size=key_size)
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
loss_fn = nn.CrossEntropyLoss()

# Обучение
epochs = 8
for epoch in range(epochs):
    total_loss = 0
    total_accuracy = 0

    for imgs, labels in tqdm(train_loader):
        optimizer.zero_grad()

```

```
# Генерируем корректный ключ
correct_key = torch.ones((imgs.size(0), key_size)) * 0.5

preds = model(imgs, correct_key)
loss = loss_fn(preds, labels)
loss.backward()
optimizer.step()

total_loss += loss.item()
total_accuracy += accuracy(preds, labels)

avg_loss = total_loss / len(train_loader)
avg_accuracy = total_accuracy / len(train_loader)

print(f'Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, Accuracy: {avg_accuracy:.4f}')
```